



Computer Systems Security

Dr. Sandra Wahid

Public-Key Cryptosystems

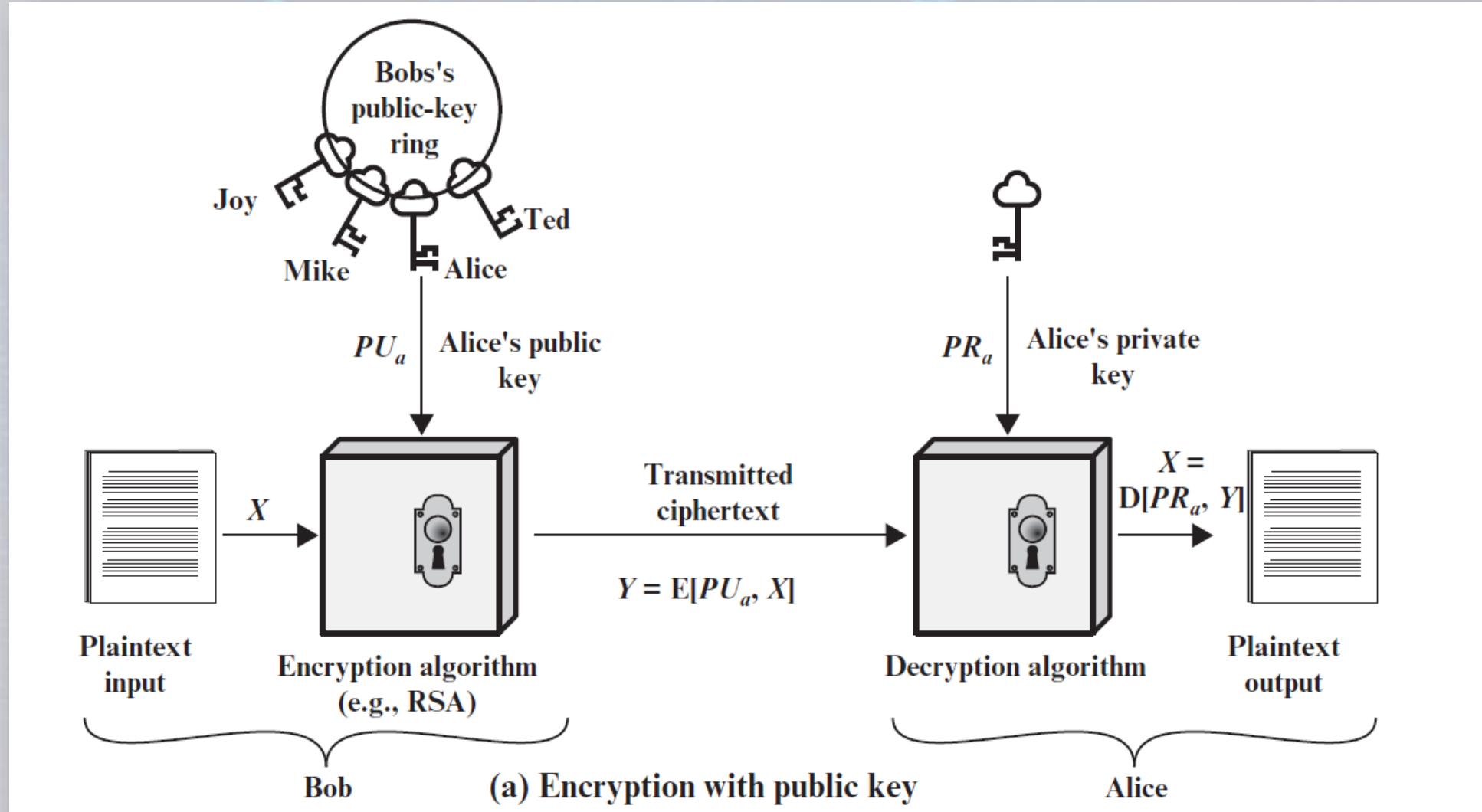
- Public-key algorithms are based on **mathematical functions** rather than on substitution and permutation.
- Public-key cryptography is **asymmetric**, involving the use of **two separate keys**, in contrast to symmetric encryption, which uses only one key.
- One misconception is that public-key encryption is more secure from cryptanalysis than is symmetric encryption.
 - In fact, the security of any encryption scheme depends on the *length of the key* and the *computational work* involved in breaking a cipher.
- A second misconception is that public-key encryption is a general-purpose technique that has made symmetric encryption obsolete.

Public-Key Cryptosystems

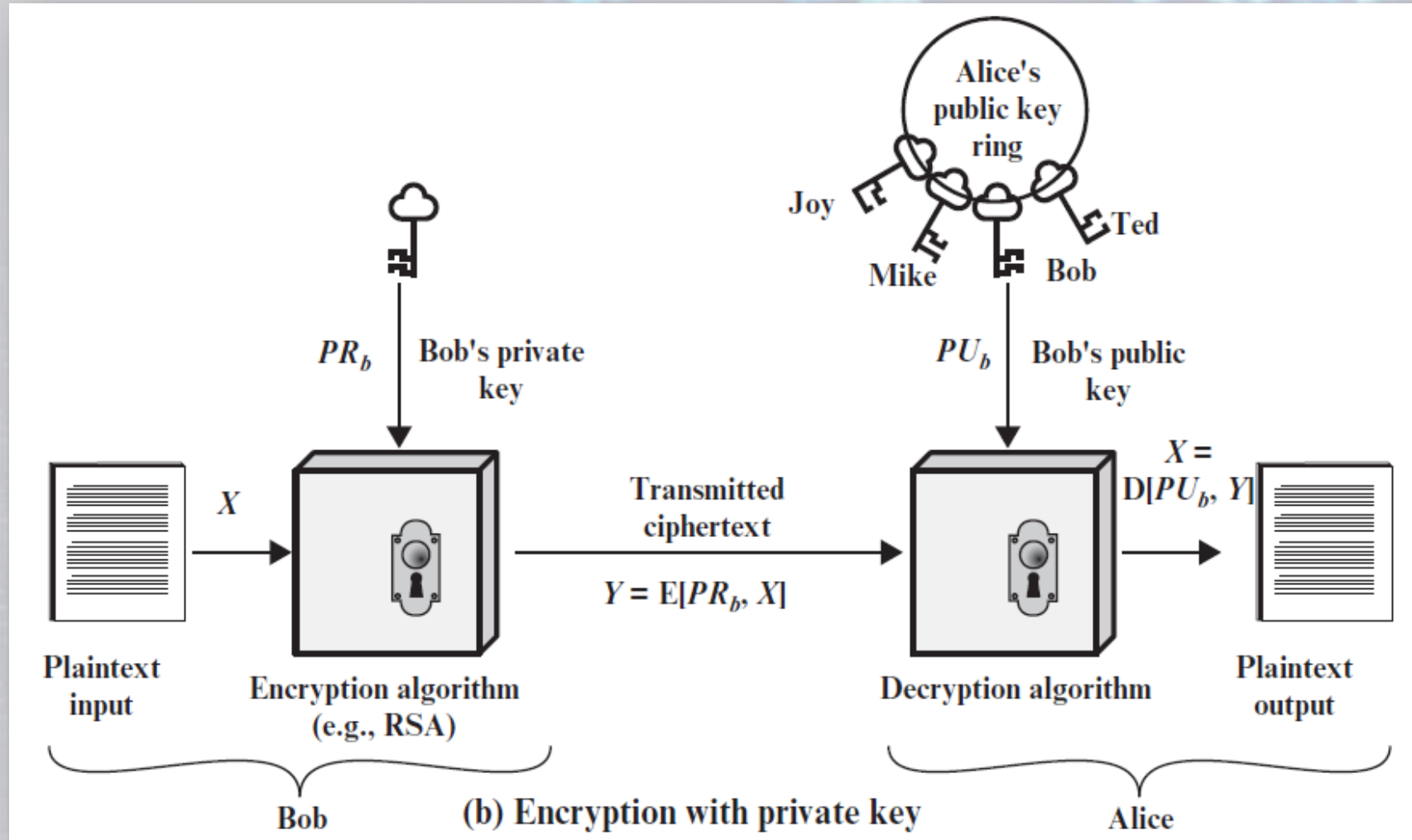
- Public-key/two-key/asymmetric cryptography involves the use of two keys:
 - a **public-key**, which may be known by anybody, and can be used to **encrypt** messages, and **verify signatures**.
 - a related **private-key**, known only to the recipient, used to **decrypt** messages, and **sign (create) signatures**.
- It is **infeasible** to determine private key from public.
- It is asymmetric because those who encrypt messages or verify signatures cannot decrypt messages or create signatures.
- Applications for Public-Key Cryptosystems:
 - **Encryption/decryption**: The sender encrypts a message with the recipient's public key, and the recipient decrypts the message with the recipient's private key.
 - **Digital signature**: The sender "signs" a message with its private key. Signing is achieved by a cryptographic algorithm applied to the message or to a small block of data that is a function of the message.
 - **Key exchange**: Two sides cooperate to exchange a session key, which is a secret key for symmetric encryption generated for use for a particular transaction (or session) and valid for a short period of time.

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Encryption/Confidentiality



Digital Signature/Authentication



- Because the message was encrypted using B's private key, only B could have prepared the message.
- Therefore, the entire encrypted message serves as a digital signature.
- In addition, it is impossible to alter the message without access to B's private key, so the message is authenticated both in terms of source and in terms of data integrity.

Comparison

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if the key is kept secret. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and a related algorithm for decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if one of the keys is kept secret. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

Requirements for Public-Key Cryptography

1. It is computationally easy for a party B to generate a key pair (public key PU_b , private key PR_b).
2. It is computationally easy for a sender A, knowing the public key and the message to be encrypted, M , to generate the corresponding ciphertext:

$$C = E(PU_b, M)$$

3. It is computationally easy for the receiver B to decrypt the resulting ciphertext using the private key to recover the original message:

$$M = D(PR_b, C) = D[PR_b, E(PU_b, M)]$$

4. It is computationally infeasible for an adversary, knowing the public key, PU_b , to determine the private key, PR_b .
5. It is computationally infeasible for an adversary, knowing the public key, PU_b , and a ciphertext, C , to recover the original message, M .
6. Not necessary for all: The two keys can be applied in either order:

$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)]$$

Requirements for Public-Key Cryptography

- **One-way function:** is one that maps a domain into a range such that every function value has a unique inverse, with the condition that the calculation of the function is easy, whereas the calculation of the inverse is infeasible:

$$\begin{array}{ll} Y = f(X) & \text{easy} \\ X = f^{-1}(Y) & \text{infeasible} \end{array}$$

- **Trap-door one-way function:** which is easy to calculate in one direction and infeasible to calculate in the other direction unless certain additional information is known → with the additional information the inverse can be calculated in polynomial time:

$$\begin{array}{ll} Y = f_k(X) & \text{easy, if } k \text{ and } X \text{ are known} \\ X = f_k^{-1}(Y) & \text{easy, if } k \text{ and } Y \text{ are known} \\ X = f_k^{-1}(Y) & \text{infeasible, if } Y \text{ is known but } k \text{ is not known} \end{array}$$

- Thus, the development of a practical public-key scheme depends on discovery of a suitable trap-door one-way function.

RSA

- Rivest-Shamir-Adleman (RSA) scheme is the most widely accepted and implemented general-purpose approach to public-key encryption.
- Plaintext and ciphertext are integers between 0 and $n-1$ for some n .
 - A typical size for n is 1024 bits $\rightarrow n$ is less than 2^{1024} .
- To encrypt a message M , the sender:
 - obtains **public key** of recipient $PU=\{e, n\}$
 - computes: $C = M^e \bmod n$ where $0 \leq M < n$
- To decrypt the ciphertext C , the owner:
 - uses their **private key** $PR=\{d, n\}$
 - computes: $M = C^d \bmod n$
- Note that the message M must be smaller than the modulus n (use blocks if needed).

RSA

- For this algorithm to be satisfactory for public-key encryption, the following requirements must be met:

- It is possible to find values of e, d , and n such that $M^{ed} \bmod n = M$ for all $M < n$. $\rightarrow M = C^d \bmod n = (M^e)^d \bmod n = M^{ed} \bmod n$
- It is relatively easy to calculate $M^e \bmod n$ and $C^d \bmod n$ for all values of $M < n$.
- It is infeasible to determine d given e and n .

- $M^{ed} \bmod n = M$ can be achieved if e and d are multiplicative inverses modulo $\phi(n)$.

- For p, q prime: $\phi(pq) = (p - 1)(q - 1)$

- $ed \bmod \phi(n) = 1 \rightarrow \begin{matrix} ed \equiv 1 \bmod \phi(n) \\ d \equiv e^{-1} \bmod \phi(n) \end{matrix}$

$$(ed) \bmod \phi(n) = 1$$

- (ed) and 1 are congruent modulo $\phi(n)$
 $\rightarrow (ed) \bmod \phi(n) = 1 \bmod \phi(n) = 1$
- d and e^{-1} are congruent modulo $\phi(n)$
 $\rightarrow d \bmod \phi(n) = e^{-1} \bmod \phi(n)$

- This is true only if d (and therefore e) is relatively prime to $\phi(n)$: $\gcd(\phi(n), d) = 1$

RSA

Key Generation by Alice

Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \times q$	
Calculate $\phi(n) = (p - 1)(q - 1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$d \equiv e^{-1} \pmod{\phi(n)}$
Public key	$PU = \{e, n\}$
Private key	$PR = \{d, n\}$

Select $\equiv$ choose randomly

Encryption by Bob with Alice's Public Key

Plaintext:	$M < n$
Ciphertext:	$C = M^e \pmod n$

Decryption by Alice with Alice's Private Key

Ciphertext:	C
Plaintext:	$M = C^d \pmod n$

1. Select two prime numbers, $p = 17$ and $q = 11$.
2. Calculate $n = pq = 17 \times 11 = 187$.
3. Calculate $\phi(n) = (p - 1)(q - 1) = 16 \times 10 = 160$.
4. Select e such that e is relatively prime to $\phi(n) = 160$ and less than $\phi(n)$; we choose $e = 7$.
5. Determine d such that $de \equiv 1 \pmod{160}$ and $d < 160$. The correct value is $d = 23$, because $23 \times 7 = 161 = (1 \times 160) + 1$; d can be calculated using the extended Euclid's algorithm

Extended Euclidean Algorithm

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

➤ Euclidean Algorithm to compute GCD(a,b) is:
 Euclid(a,b)
 if (b=0) then return a;
 else return Euclid(b, a mod b);

$$ax + by = d = \gcd(a, b)$$

x and y will have opposite signs

Extended Euclidean Algorithm			
Calculate	Which satisfies	Calculate	Which satisfies
$r_{-1} = a$		$x_{-1} = 1; y_{-1} = 0$	$a = ax_{-1} + by_{-1}$
$r_0 = b$		$x_0 = 0; y_0 = 1$	$b = ax_0 + by_0$
$r_1 = a \bmod b$ $q_1 = \lfloor a/b \rfloor$	$a = q_1b + r_1$	$x_1 = x_{-1} - q_1x_0 = 1$ $y_1 = y_{-1} - q_1y_0 = -q_1$	$r_1 = ax_1 + by_1$
$r_2 = b \bmod r_1$ $q_2 = \lfloor b/r_1 \rfloor$	$b = q_2r_1 + r_2$	$x_2 = x_0 - q_2x_1$ $y_2 = y_0 - q_2y_1$	$r_2 = ax_2 + by_2$
$r_3 = r_1 \bmod r_2$ $q_3 = \lfloor r_1/r_2 \rfloor$	$r_1 = q_3r_2 + r_3$	$x_3 = x_1 - q_3x_2$ $y_3 = y_1 - q_3y_2$	$r_3 = ax_3 + by_3$
⋮	⋮	⋮	⋮
$r_n = r_{n-2} \bmod r_{n-1}$ $q_n = \lfloor r_{n-2}/r_{n-1} \rfloor$	$r_{n-2} = q_nr_{n-1} + r_n$	$x_n = x_{n-2} - q_nx_{n-1}$ $y_n = y_{n-2} - q_ny_{n-1}$	$r_n = ax_n + by_n$
$r_{n+1} = r_{n-1} \bmod r_n = 0$ $q_{n+1} = \lfloor r_{n-1}/r_n \rfloor$	$r_{n-1} = q_{n+1}r_n + 0$		$d = \gcd(a, b) = r_n$ $x = x_n; y = y_n$

If $d=r_n=1$ then $y= b^{-1} \bmod a$

Extended Euclidean Algorithm

- Example:

Extended Euclidean Algorithm Example $a = 1759$ $b = 550$

i	r_i	q_i	x_i	y_i
-1	1759		1	0
0	550		0	1
1	109	3	1	-3
2	5	5	-5	16
3	4	21	106	-339
4	1	1	-111	355
5	0	4		

Result: $d = 1$; $x = -111$; $y = 355$

Since $d=1$ then 355 is $550^{-1} \bmod 1759$

In RSA, apply Extended Euclidean Algorithm for $a=\phi(n)$ and $b=e \rightarrow$ we are sure that $\gcd(\phi(n), e) = 1$ then $y = e^{-1} \bmod \phi(n) = d \bmod \phi(n) = d$ since $d < \phi(n)$

RSA

- $PU = \{7, 187\}$ and $PR = \{23, 187\}$,
- $M = 88 \rightarrow < 187$
- Exponentiation in modular arithmetic:

$$[(a \bmod n) \times (b \bmod n)] \bmod n = (a \times b) \bmod n$$

For encryption, we need to calculate
 $C = 88^7 \bmod 187$

$$\begin{aligned} 88^7 \bmod 187 &= [(88^4 \bmod 187) \times (88^2 \bmod 187) \\ &\quad \times (88^1 \bmod 187)] \bmod 187 \\ 88^1 \bmod 187 &= 88 \\ 88^2 \bmod 187 &= 7744 \bmod 187 = 77 \\ 88^4 \bmod 187 &= 59,969,536 \bmod 187 = 132 \\ 88^7 \bmod 187 &= (88 \times 77 \times 132) \bmod 187 = 894,432 \bmod 187 = 11 \end{aligned}$$

For decryption, we need to calculate
 $M = 11^{23} \bmod 187$

$$\begin{aligned} 11^{23} \bmod 187 &= [(11^1 \bmod 187) \times (11^2 \bmod 187) \times (11^4 \bmod 187) \\ &\quad \times (11^8 \bmod 187) \times (11^8 \bmod 187)] \bmod 187 \\ 11^1 \bmod 187 &= 11 \\ 11^2 \bmod 187 &= 121 \\ 11^4 \bmod 187 &= 14,641 \bmod 187 = 55 \\ 11^8 \bmod 187 &= 214,358,881 \bmod 187 = 33 \\ 11^{23} \bmod 187 &= (11 \times 121 \times 55 \times 33 \times 33) \bmod 187 \\ &= 79,720,245 \bmod 187 = 88 \end{aligned}$$

Represent exponent in its binary format and write it as sum of its power of two numbers, (N.B.: can do shortcuts)

RSA Security

- Possible approaches to attacking RSA are:
 - **Brute force:** This involves trying all possible private keys.
 - **Mathematical attacks:** There are several approaches, all equivalent in effort to factoring the product of two primes.
 - **Timing attacks:** These depend on the running time of the decryption algorithm.
 - **Hardware fault-based attack:** This involves inducing hardware faults in the processor that is generating digital signatures.
 - **Chosen ciphertext attacks:** This type of attack exploits properties of the RSA algorithm.

The Factoring Problem

- We can identify three approaches to attacking RSA mathematically:

1. Factor n into its two prime factors. This enables calculation of $\phi(n) = (p - 1) \times (q - 1)$, which in turn enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
2. Determine $\phi(n)$ directly, without first determining p and q . Again, this enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
3. Determine d directly, without first determining $\phi(n)$.

cryptanalysis of RSA have focused on the task of factoring n into its two prime factors.

- Remember the following:

Modular exponentiation is easy: Given n , m , and e , it's easy to compute $c = m^e \bmod n$.

Modular root extraction – the reverse of modular exponentiation – is easy given the prime factors: Given n , e , c , and the prime factors p and q , it's easy to recover the value m such that $c = m^e \bmod n$.

Modular root extraction is otherwise hard: Given only n , e , and c , but not the prime factors, it appears to be quite hard to recover the value m .



Thank You